Πολυτεχνείο Κρήτης
Σχολή ΗΜΜΥ
Δομές Δεδομένων και Αλγόριθμοι

Αλεξίου Σταμάτιος

Α.Μ: 2020030158

Παράδοση αναφοράς 1ης εργασίας
Ημερομηνία Παράδοσης: 24 Μαρτίου 2023

Περιεχόμενα

1 Σύντομη περιγραφή του κώδικα **3**

2 Αποτελέσματα μετρήσεων **6**

3 Σχολιασμός Αποτελεσμάτων και Διαγράμματα **8**

1 Σύντομη περιγραφή του κώδικα

Ο κώδικας αποτελείται από 2 packages: το **implementation** και το **structure**. Το structure αφορά την δομή και την υλοποίηση του κώδικα , ενώ το implementation αφορά τις πειραματικές μετρήσεις που έπρεπε να πραγματοποιήσουμε. Το **structure** περιέχει 4 κλάσεις οι οποίες περιγράφονται παρακάτω.

- **DataClass** : Αποτελεί την κύρια κλάση , καθώς θα διαχειριζόμαστε τα δεδομένα της. Περιέχει έναν ακέραιο αριθμό-κλειδί (int key) και τα δεδομένα της τα οποία είναι μια συμβολοσειρά (String data). Η κλάση εκτός από getters και setters περιέχει 2 ακόμα συναρτήσεις:

  (1) public byte[] convertToBytes() : Μεταρέπει μια DataClass σε array από bytes και επιστρέφει το array.

  (2) public static int[] convertToInstance(RandomAccessFile file,int page,DataPage dp) : Διαβάζει μια σελίδα (page) που της δίνεται ως όρισμα από ένα file και επιστρέφει ένα array με τα κλειδιά των Instances στην σελίδα αυτή.

- **AccessCounter** : Η κλάση αυτή , η οποία έχει υλοποιηθεί στο φροντηστήριο θα χρησιμεύσει στο να μετρήσουμε τον μέσο αριθμό προσβάσεων στο δίσκο για κάθε διαφορετική πειραματική μέτρηση που θα διεξάγουμε. Θέλουμε να έχουμε 1000 μετρητές γιατί κάθε φορά αναζητούμε 1000 κλειδιά , οπότε θα αποθηκεύουμε τον αριθμό προσβάσεων στο δίσκο για το κάθε κλειδί σε ξεχωριστό μετρητή. Περιέχει μεθόδους ώστε να κάνουμε reset τους μετρητές, να αυξάνουμε ένα μετρητή κατά ένα ή όσο θέλουμε εμείς , καθώς και μία μέθοδο όπου επιστρέφει τον αριθμό που είναι αποθηκευμένος σε ένα μετρητή.

- **Pairs** : Η κλάση αυτή περιέχει δύο ακαίρεους αριθμούς , ένα κλειδί (int key) και ένα δείκτη στη σελίδα όπου βρίσκεται το κλειδί (int index). Τα ζεύγη αυτά θα αποθηκευτούν σε άλλο αρχείο από τις DataClasses. Η κλάση υλοποιεί το Comparable interface οπότε έχουμε και την μέθοδο compareTo όπου συγκρίνει δύο Pairs ως προς το κλειδί το οποίο θα χρειαστεί στο (γ) μέρος της άσκησης για την ταξινόμηση των ζευγαριών.Η κλάση εκτός από getters και setters περιέχει 2 ακόμα συναρτήσεις:

  (1) public byte[] convertToBytes() : Μεταρέπει ένα στιγμιότυπο της κλάσης Pairs σε array από bytes και επιστρέφει το array.

  (2) public static int[] convertToInstance(RandomAccessFile file,int page,DataPage dp) : Διαβάζει μια σελίδα (page) που της δίνεται ως όρισμα από ένα file και επιστρέφει ένα array με τα κλειδιά και τους δείκτες των Instances στην σελίδα αυτή.

- **DataPage** : Αποτελεί την βασικότερη κλάση καθώς θα αποθηκεύουμε στο δίσκο με μορφή σελίδων. Η κλάση εκτός από getters και setters περιέχει και αρκετές ακόμα συναρτήσεις:

  (1) public void randomWriteOnDisk(RandomAccessFile file,byte[] array) :Η συνάρτηση αυτή αφορά την εγγραγή DataClass στο δίσκο και γράφει στο αρχείο (file) το array από bytes που της δίνεται ως όρισμα , μετά από ελέγχους (Αν έχει γεμίσει η σελίδα ή αν πρόκειται για την τελευταία σελίδα). Τέλος, αν γίνει η εγγραφή καθαρίζουμε τον buffer της σελίδας.

(2) public static int[] randomReadFromDisk(int key,int[] keys) : Παίρνει ως όρισμα ένα ακέραιο κλειδί και τον ψάχνει στο array από κλειδιά που ουσιαστικά είναι η τιμή επιστροφής της συνάρτησης convertToInstance στην κλάση DataClass.Αν το βρει επιστρέφει το κλειδί από το array που έχει την ίδια τιμή με το αρχικό κλειδί αλλιώς επιστρέφει την ελάχιστη ακέραια τιμή.

(3) public void indexedWriteOnDisk(RandomAccessFile file,byte[] array) :Η συνάρτηση αυτή αφορά την εγγραγή Pairs στο δίσκο και γράφει στο αρχείο (file) το array από bytes που της δίνεται ως όρισμα , μετά από ελέγχους (Αν έχει γεμίσει η σελίδα ή αν πρόκειται για την τελευταία σελίδα). Τέλος, αν γίνει η εγγραφή καθαρίζουμε τον buffer της σελίδας.

(4) public int[] indexedUnsortedReadFromDisk(int key, int[] info) : Παίρνει ως όρισμα ένα ακέραιο κλειδί και το ψάχνει στο array info που αποτελείται από κλειδιά και θέσεις των κλειδιών στο δίσκο. Αν βρεθεί το κλειδί τότε επιστρέφει ένα array με το κλειδί και την θέση του στο αρχείο, αλλιώς επιστρέφει null.

(5) public int indexedSortedReadFromDisk(int key, int[] info):Παίρνει ως όρισμα ένα ακέραιο κλειδί και το ψάχνει στο array info που αποτελείται από κλειδία και θέσεις των κλειδιών στο δίσκο. Αν βρεθεί το κλειδί τότε επιστρέφεται ή τιμή 1, αν δεν βρεθεί και το πρώτο κλειδί στο array είναι μεγαλύτερο από το κλειδί που ψάχνουμε τότε επιστρέφεται η τιμή 2 , αλλιώς αν το τελευταίο κλειδί στο array είναι μικρότερο από το κλειδί που ψάχνουμε επιστρέφεται η τιμή 3.Διαφορετικά, επιστρέφουμε -1.

Έπειτα υπάρχουν και άλλες συναρτήσεις οι οποίες μας βοηθούν να καταλάβουμε πότε μια σελίδα είναι γεμάτη , πότε μια σελίδα είναι η τελευταία , πόσα Instances χωράνε σε μια σελίδα κτλ.

Το **implementation** περιέχει μία κλάση, την κλάση :

- **Tests** : Η κλάση αυτή περιέχει την main , όπου δημιουργούμε 3 φιλες και εκτελούμε τις πειραματικές μετρήσεις για κάθε αριθμό records και μεγέθους datasize. Οι μετρήσεις γίνονται μέσα στην συνάρτηση :

(1) public static void saveReadAndCalculate(int records ,int dataSize, RandomAccessFile file1,RandomAccessFile file2,RandomAccessFile file3) : Στη συνάρτηση αυτή αρχικοποιούνται οι κλάσεις DataClass, Pairs και δημιουργούμε και μία DataPage.Αρχικά, γράφουμε τις κλάσεις DataClass, Pairs στο δίσκο σε ξεχωριστά φιλες. Στην συνέχεια, ανάλογα με τον αριθμό των records δημιουργούμε 1000 μοναδικά (ή όχι αν records ˚ 1000) και τα αναζητούμε στην κάθε μία περίπτωση.Επίσης , μετράμε τον χρόνο που παίρνει η αναζήτηση στο αρχείο . Αφού τελειώσει η διαδικασία τυπώνουμε τα στατιστικά στην κονσόλα (μέσος αριθμός προσβάσεων στο δίσκο και μέσο χρόνο). Όταν, ολοκληρωθούν οι πρώτες 2 διαδικασίες , τότε ταξινομούμε τα Pairs και τα γράφουμε σε ένα 3ο αρχείο και πραγματοποιούμε τις μετρήσεις μας και εκεί. Στο τέλος, τυπώνουμε πάλι τα στατιστικά.

(2) public static int randomSearchKey(DataPage dp,int key ,RandomAccessFile file,int count) : Αφορά τον πρώτο τρόπο οργάνωσης και ψάχνει σειριακά στο αρχείο ,ανά σελίδα, για να βρει το κλειδί που της δίνεται ως όρισμα.Κάθε ανάγνωσης σελίδας αυξάνει και

τον μετρητή στην θέση count κατά 1. Αν το κλειδί βρεθεί, επιστρέφεται το κλειδί αλλιώς επιστρέφεται -1.

(3) public static int indexedUnsortedSearchKey(DataPage dp,int key ,RandomAccessFile file1,RandomAccessFile file2,int count) : Αφορά τον δεύτερο τρόπο οργάνωσης και ψάχνει σειριακά στο αρχείο2 ,ανά σελίδα, για να βρει το κλειδί που της δίνεται ως όρισμα.Κάθε ανάγνωση σελίδας αυξάνει και τον μετρητή στην θέση count κατά 1. Αν το κλειδί βρεθεί, παίρνουμε τον αριθμό σελίδας του στο αρχείο 1 και ψάχνουμε το κλειδί αυτό στην συγκεκριμένη σελίδα στο αρχείο 1 , αυξάνοντας ακόμη μια φορά τον μετρητή στην θέση count κατά 1 και επιστρέφουμε την τιμή του κλειδιού. Αν το κλειδί δεν υπάρχει επιστρέφουμε -1.

(4) public static int binarySearch(DataPage dp,int leftIndex, int rightIndex, int key, int count, RandomAccessFile file1, RandomAccessFile file2) : Αφορά τον τρίτο τρόπο οργάνωσης και ψάχνει στο αρχείο με τα Pairs τα οποία είναι ταξινομημένα με βάση το κλειδί, εφαρμόζοντας βιναρψΣεαρςη αυξάνοντας κάθε φορά τον μετρητή στην θέση count κατά 1. Η δομή του κώδικα έχει υλοποιηθεί σε φροντιστήριο του μαθήματος.Αν βρεθεί το κλειδί παίρνουμε τον αριθμό σελίδας του στο αρχείο 1 και ψάχνουμε το κλειδί αυτό στην συγκεκριμένη σελίδα , αυξάνοντας ακόμη μια φορά τον μετρητή στην θέση count κατά 1 και επιστρέφουμε την τιμή του κλειδιού. Αν το κλειδί δεν υπάρχει επιστρέφουμε την ελάχιστη ακέραια τιμή.

(5) public static int calculateMaxPagesForIndexedFile(int records,DataPage dp): Η συνάρτηση αυτή υπολογίζει την τελευταία σελίδα στο αρχείο με τα ζεύγη, ώστε να περαστεί ως όρισμα στο δεξί άκρο του binarySearch.

(6) public static void printStats(long time[],int records,String opt): Η συνάρτηση αυτή εκτυπώνει στην κονσόλα τους μέσους αριθμούς προσβάσεων στο δίσκο και τον μέσο χρόνο αναζήτησης. Τέλος , "καθαρίζει" τους μετρητές χρόνου και προσβάσεων.

(7) public static long averageTimeSpentOnSearching(long[] time): Η συνάρτηση αυτή υπολογίζει και επιστρέφει τον μέσο χρόνο αναζήτησης.

(8) public static String averageAccessOnSearching(): Η συνάρτηση αυτή υπολογίζει και επιστρέφει τον μέσο αριθμό προσβάσεων στο δίσκο με ακρίβεια 1 δεκαδικό ψηφίο.

Τέλος υπάρχουν υλοποιημένες συναρτήσεις για την παραγωγή τυχαίων μοναδικών ή όχι αριθμών και για την παραγωγή τυχαίου αλφαριθμητικού String οι οποίες μας δίνονταν έτοιμες στην εκφώνηση της άσκησης.

2 Αποτελέσματα μετρήσεων

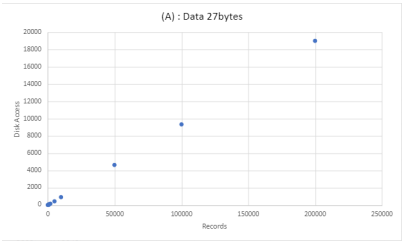
Τρέχοντας των κώδικα παίρνουμε τα εξής αποτελέσματα τα οποία φαίνονται παρακάτω σε μορφή πινάκων.

	Τρόπος Α	Τρόπος Α	Τρόπος Β	Τρόπος Β	Τρόπος Γ	Τρόπος Γ
N	Χρόνος ανά αναζήτηση Μέγεθος δεδομένων 55 bytes	Χρόνος ανά αναζήτηση Μέγεθος δεδομένων 27 bytes	Χρόνος ανά αναζήτηση Μέγεθος δεδομένων 55 bytes	Χρόνος ανά αναζήτηση Μέγεθος δεδομένων 27 bytes	Χρόνος ανά αναζήτηση Μέγεθος δεδομένων 55 bytes	Χρόνος ανά αναζήτηση Μέγεθος δεδομένων 27 bytes
50	69425ns	42656ns	15782ns	19651ns	15785ns	15873ns
100	122219ns	57461ns	22750ns	23534ns	20170ns	19465ns
200	198596ns	120348ns	36319ns	35244ns	21487ns	22432ns
500	482553ns	279670ns	73468ns	75396ns	23343ns	27227ns
800	757988ns	407420ns	115899ns	118408ns	28398ns	31769ns
1000	958665ns	532129ns	142428ns	143159ns	32622ns	28943ns
2000	1909448	1019836	294245ns	299467ns	36168ns	42070ns
5000	4847693	2449642	696023ns	707399ns	42912ns	43623ns
10000	9615308	4877655	1379372	1379051ns	53754ns	49014ns
50000	53099979	25333544	7992478	7999782	76480ns	75841ns
100000	115516017	55198067	15822548	15526398	85489ns	78337ns
200000	199215138	101511932	28034946	27369950	88991ns	77803ns

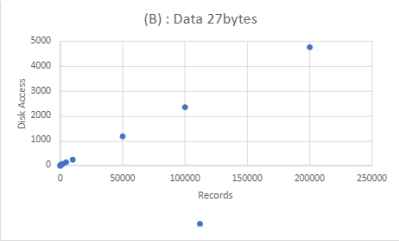
	Τρόπος Α	Τρόπος Α	Τρόπος Β	Τρόπος Β	Τρόπος Γ	Τρόπος Γ
Ν	Αριθμός προσ-βάσεων ανά αναζήτηση Μέγε-θος δε-δομένων 55 bytes	Αριθμός προσ-βάσεων ανά αναζήτηση Μέγε-θος δε-δομένων 27 bytes	Αριθμός προσ-βάσεων ανά αναζήτηση Μέγε-θος δε-δομένων 55 bytes	Αριθμός προσ-βάσεων ανά αναζήτηση Μέγε-θος δε-δομένων 27 bytes	Αριθμός προσ-βάσεων ανά αναζήτηση Μέγε-θος δε-δομένων 55 bytes	Αριθμός προσ-βάσεων ανά αναζήτηση Μέγε-θος δε-δομένων 27 bytes
50	9,9	5,4	2,2	2,2	2,2	2,1
100	18,7	9,9	3,5	3,6	2,3	2,4
200	37,6	19,0	5,8	5,8	3,3	3,2
500	93,4	47,4	12,6	12,7	4,0	4,1
800	146,8	74,8	19,1	19,4	4,6	4,6
1000	184,2	94,9	24,1	24,8	5,2	5,0
2000	372,2	189,1	47,5	48,2	6,3	6,0
5000	938,1	469,6	118,3	118,5	7,3	7,4
10000	1856,8	934,0	233,1	234,4	8,3	8,2
50000	9351,1	4669,7	1169,9	1168,4	10,5	10,2
100000	19070,1	9325,5	2384,5	2332,1	11,8	11,6
200000	37506,6	19016,3	4689,1	4754,8	12,4	12,8

3 Σχολιασμός Αποτελεσμάτων και Διαγράμματα

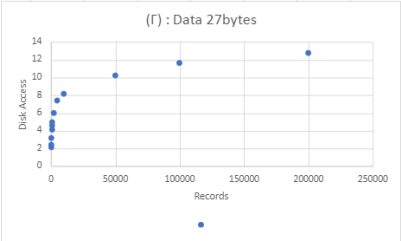
Με βάση τους παραπάνω πίνακες φτιάξαμε τα παρακάτω διαγράμματα όπου στον X άξονα είναι το πλήθος στιγμιοτύπων, και στον Υ άξονα ο μέσος αριθμός προσβάσεων και ο μέσος χρόνος εκτέλεσης για μία αναζήτηση.



(a)

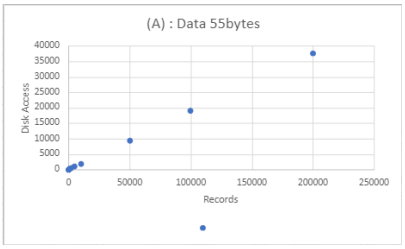


(b)

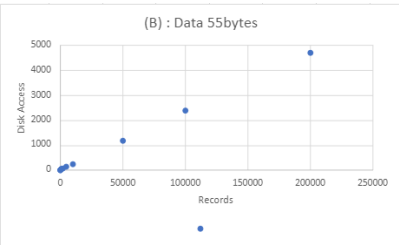


(c)

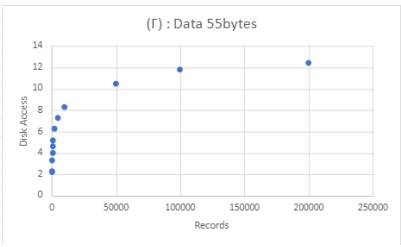
Disk Access on Data 27Bytes



(a)

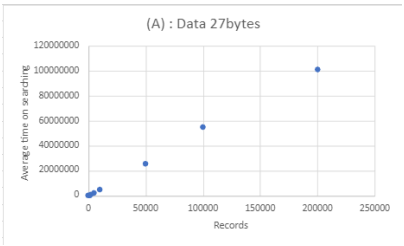


(b)

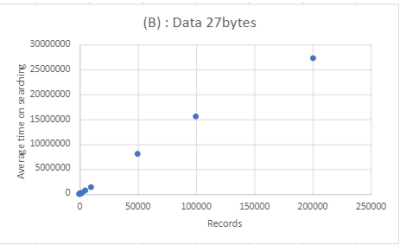


(c)

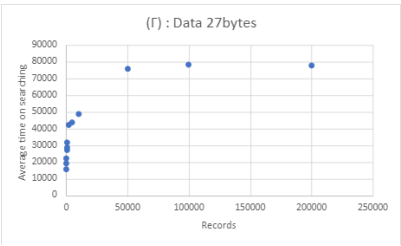
Disk Access on Data 55Bytes



(a)

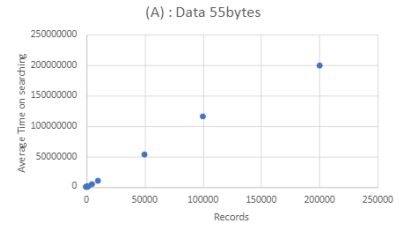


(b)

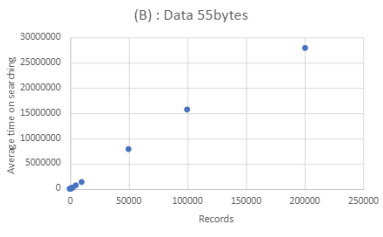


(c)

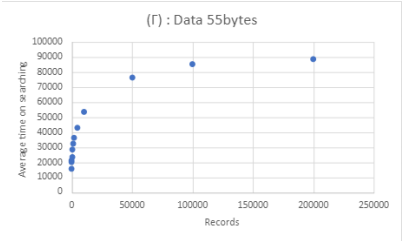
Average Time On Searching, Data 27Bytes



(a)



(b)



(c)

Average Time On Searching, Data 55Bytes

Παρατηρούμε ότι τα διαγράμματα του χρόνου και των προσβάσεων στο δίσκο στον πρώτο και δεύτερο τρόπο οργάνωσης, ανεξάρτητα από το μέγεθος των δεδομένων , ακολουθούν γραμμική μορφή , πράγμα το οποίο είναι αναμενόμενο αφού διαβάζουμε σειριακά το αρχείο , ανά σελίδα μέσα σε ένα for-loop που τρέχει N φορές , όπου N ο αριθμός τον στιγμυοτύπων.Συνεπώς, όσο αυξάνεται το πλήθος των εγγραφών στο δίσκο τόσο αυξάνεται και ο χρόνος αναζήτησης ,αλλά και οι προσβάσεις στο δίσκο, αφού γνωρίζουμε από την θεωρία ότι έχουμε πολυπλοκότητα $T(n) \in O(N)$. Ωστόσο, στον δεύτερο τρόπο (σε σχέση με τον πρώτο) η καμπύλη έχει πολύ μικρότερη κλίση, καθώς το αρχείο δεικτοδότησης περιέχει πολύ λιγότερες σελίδες από το 1ο αρχείο. Συνεπώς , μπορούμε να το διαβάσουμε γρηγορότερα και με λιγότερες προσβάσεις στο δίσκο. Τέλος, όσον αφορά τον τρίτο τρόπο αναζήτησης παρατηρούμε ότι τα διαγράμματα του χρόνου και των προσβάσεων στο δίσκο, ανεξάρτητα από το μέγεθος των δεδομένων, ακολουθούν λογαριθμική μορφή, πράγμα το οποίο είναι αναμενόμενο γιατί το array από τα ζεύγη είναι ταξινομιμένο και κάνουμε αναζήτηση με binary search. Συνεπώς, απ΄όσο γνωρίζουμε από την θεωρία έχουμε πολυπλοκότητα $T(n) \in O(log(N))$.

Επιπλέον, παρατηρούμε πως ο χρόνος δεν ακολουθεί πάντα ακριβώς την ίδια αύξηση με τον αριθμό προσβάσεων στο δίσκο. Συγκεκριμένα, παρατηρούμε πως σε ορισμένες περιπτώσεις ο χρόνος μπορεί να έχει μεγαλύτερη αύξηση. Αυτό, μπορεί να οφείλεται σε διάφορους λόγους, όπως για παράδειγμα στον δίσκο μας. Αντίστοιχα, ο χρόνος μπορεί να αυξηθεί και σε μικρότερο βαθμό , γεγονός το οποίο μπορεί να οφείλεται σε prefetching , οπότε ενώ μια σελίδα μπορεί να έχει φορτωθεί νωρίτερα στην virtual memory εμείς να αυξάνουμε κανονικά τα disk access. Δεν θα πρέπει να ξεχνάμε πως από την στιγμή που κάνουμε τυχαίες αναζητήσεις δεν μπορούμε να ξέρουμε αν έχουμε Best,Average ή Worst Case σε κάθε αναζήτηση.

Συγκρίνοντας τους τρεις τρόπους οργάνωσης ερχόμαστε στο συμπέρασμα ότι ο (B) και ο (Γ) τρόπος χρειάζονται περισσότερη μνήμη στο δίσκο σε σύγκριση με τον (A) , καθώς γράφουμε ακόμη ένα αρχείο. Ωστόσο, σε περίπτωση που θέλουμε να ανακτήσουμε δεδομένα , η όλη διαδικασία απαιτεί πολύ λιγότερο χρόνο και πολύ λιγότερες προσβάσεις στο δίσκο.Συγκρίνοντας τον (B) με τον (Γ), παρατηρούμε ότι με τον (Γ) τρόπο οργάνωσης μπορούμε να ανακτήσουμε δεδονένα πολύ πιο γρήγορα , εξοικονομώντας έτσι χρόνο και προσβάσεις στο δίσκο. Βέβαια , δεν θα πρέπει να ξεχνάμε ότι το array μας θα πρέπει να είναι ταξινομιμένο (για να εφαρμόσουμε και binary search) !

Αλεξίου Σταμάτιος